SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: OPERATING SYSTEMS **COURSE CODE: CSA04**

COURSE OUTCOME COVERED

CO2: Analyze process management and deadlock handling techniques to improve CPU utilization and system performance(BL3-BL4)

Assessment Tool Weightage: 40%

QUESTIONS: 10 **Total Marks:100**

Annexure A

Analytical Problem Solving

Code of Conduct:

I Priyadharshini.KS *(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*

I understand that any violation of academic integrity rules will result in disciplinary action.

Priyadharshini

Signature of the student:

Question 1: CPU Scheduling – Waiting Time & Turnaround Time Analysis

A set of processes arrive at time 0 with the following burst times:

Process	Burst Time (ms)
P1	10
P2	4
P3	6
P4	2

a. Calculate Waiting Time and Turnaround Time for each process using SJF (Non-preemptive).
b. Compute the average waiting time and average turnaround time.
c. Interpret whether SJF improves CPU utilization compared to FCFS for this workload.

(10 Marks)

Question 2: Round Robin Scheduling – Time Quantum Impact

Consider the following processes:

Process	Burst Time (ms)
P1	20
P2	5
P3	13
P4	8

Let the time quantum = 4 ms.

a. Construct the Gantt chart for Round Robin.
b. Calculate Waiting Time and Turnaround Time of each process.
c. Analyze how changing the quantum to 10 ms would affect system performance (CPU utilization & context switching).

(10 Marks)

Question 3: Deadlock – Safety Check Using Banker's Algorithm

A system has 3 resource types (A, B, C). Total resources:
A = 10, B = 5, C = 7.

The Allocation and Maximum Need matrices are:

Allocation Matrix

Process	A	B	C
P1	0	1	0
P2	2	0	0
P3	3	0	2
P4	2	1	1
P5	0	0	2

Maximum Matrix

Process	A	B	C
P1	7	5	3
P2	3	2	2
P3	9	0	2
P4	4	2	2
P5	5	3	3

a. Calculate the Need Matrix.
b. Determine if the system is in a safe state using Banker's Algorithm.
c. Provide a valid safe sequence if it exists.

(10 Marks)

Question 4: Deadlock Detection – Resource Allocation Graph (RAG)

A system has the following resource allocations:

- P1 is holding R1 and waiting for R2
- P2 is holding R2 and waiting for R3

- P3 is holding R3 and waiting for R1

a. Draw the Resource Allocation Graph (describe in text).
b. Determine whether the system is in deadlock.
c. Recommend a strategy (prevention OR avoidance OR recovery) to resolve it and justify the method.

(10 Marks)

Question 5: CPU Utilization – Multiprogramming Level Analysis

A system has the following process behavior:

- Each process spends 20% time in CPU and 80% waiting for I/O.
- Degree of multiprogramming = 4 processes

CPU Utilization formula:

$$U = 1 - (p^n)$$

where:

- p = probability a process is waiting for I/O,
- n = number of processes in memory

a. Calculate CPU Utilization.
b. Compute CPU utilization if the degree of multiprogramming increases to 6.
c. Interpret which configuration improves system performance and why.

(10 Marks)

Question 6: Producer–Consumer Problem Using Semaphores

A bounded buffer has a capacity of 6 items. Initially, the buffer is empty. Three producer operations occur followed by two consumer operations.

a. Show the values of the semaphores (empty, full, mutex) after each operation.
b. Explain how semaphores prevent race conditions.
c. What problem occurs if the mutex semaphore is removed? Justify your answer.

(10 Marks)

Question 7: Process Synchronization – Critical Section

Two processes increment a shared variable X initially equal to 100. Each process executes the increment operation 1000 times without synchronization.

a. Explain how a race condition occurs.
b. Determine the expected value of X with proper synchronization.
c. Suggest one hardware-based and one software-based synchronization solution.

(10 Marks)

Question 8: Thread Scheduling and Performance

A program creates 40 user-level threads on a system having 8 CPU cores. Each thread requires 12 ms CPU time and performs negligible I/O.

a. Compare the Many-to-One and One-to-One multithreading models.
b. Estimate how CPU utilisation differs between the two models.
c. Recommend the better threading model with justification.

(10 Marks)

Question 9: Monitor Synchronization

A monitor controls access to a shared printer used by multiple processes.

a. Explain how monitors ensure mutual exclusion.
b. Describe the role of condition variables wait() and signal().
c. Explain what may happen if signal() is omitted after a print job completes.

(10 Marks)

Question 10: Deadlock Prevention vs Avoidance

A system has four processes competing for two reusable resources.

a. Explain how deadlock prevention can eliminate one of the four necessary conditions for deadlock.
b. Using a suitable example, explain how deadlock avoidance differs from prevention.
c. Compare prevention and avoidance in terms of resource utilisation and system performance.

(10 Marks)

RUBRICS FOR CALCULATION: Analytical Problem Solving

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	20%	Clearly interprets problem, identifies all parameters and requirements accurately	Understands problem with minor gaps	Partial understanding; misses key elements	Misinterprets or unable to understand problem
Method Selection	20%	Selects most appropriate method/formula with strong justification	Selects correct method with minor errors	Inappropriate or partially correct method	Unable to select method
Solution Process	25%	Logical, systematic steps with correct approach throughout	Mostly correct steps with minor mistakes	Disorganized steps; multiple errors	No clear process
Accuracy of Calculation	20%	All calculations accurate; correct final answer	Minor calculation errors	Frequent errors; incorrect final answer	No valid calculations
Interpretation of Results	15%	Clearly interprets results with units and engineering relevance	Basic interpretation with minor omissions	Limited or unclear interpretation	No interpretation

Marks Evaluation:

Question No.	Problem Understandin (20%)	Method Selection(20%)	Solution Process(25%)	Accuracy of Calculation (20%)	Interpretation of Results (15%)	Total Marks (100)
Q1						
Q2						
Q3						
Q4						
Q5						
Q6						

Q7						
Q8						
Q9						
Q10						
					Overall Total (100)	

Faculty In-charge	Course Coordinator	Head of Department
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

Name : Priyadharshini
Reg : 192511248
Sub : CSA0404
Code : Operating Systems

CPU Scheduling - Waiting time & Turnaround Time :

Process	Burst time
P1	10
P2	4
P3	6
P4	2

a) Non-preemptive

Shortest Job First (SJF) is CPU scheduling algorithm that selects process with smallest burst time first. It minimizes average waiting time and turnaround time.

Gantt chart :

P4	P2	P3	P1
0	2	6	12	22

Waiting time :

WT = Start Time - Arrival Time

Process	Start Time	Waiting Time
P4	0	0
P2	2	2
P3	6	6
P1	12	12

Turnaround time :

TAT = Completion time - Arrival Time

Process	TAT
P4	2
P2	6
P3	12
P1	22

b) Average :

Average waiting time : $\frac{0+2+6+12}{4} = \frac{20}{4} = 5ms$

Average TAT : $\frac{2+6+12+22}{4} = \frac{42}{4} = 10.5$

c) Interpretation :

SJF schedules shortest process first, reducing waiting time of smaller jobs.

2) Round Robin scheduling - Time Quantum :

Process	Burst Time
P1	20
P2	5
P3	13
P4	8

chart:

P2	P3	P4	P1	P2	P3	P4	P1	P3	P1	P3	P1
0	8	12	16	20	21	25	29	33	37	38	42

TAT = Completion Time - Arrival Time

P1 → 46

P2 → 21

P3 → 42

P4 → 29

Average TAT : $\frac{138}{4}$ = 34.5 ms

WT = TAT - Burst Time

P1 → 26

P2 → 16

P3 → 29

P4 → 21

Average WT = $\frac{92}{4}$ = 23 ms

c) Effect of Increasing Time Quantum to 10 ms:

Number of context switches decrease

CPU speeds less time

CPU utilization improves.

3) Deadlock - Safety check using Banker's Algorithm

a) Need Matrix:

Need = Maximum - Allocation

P1 → 7 4 3
P2 → 1 2 2
P3 → 6 0 0
P4 → 2 1 1
P5 → 5 3 1

b) Safe state check:

Available = 3 3 2

P2 : 1 2 2 ≤ 3 3 2
P4 : 7 4 3
P1 : 7 5 5

Since all process complete successfully System is in a safe state.

c) Safe sequence:

P2 → P4 → P5 → P1 → P3

4) Deadlock Detection - Resource Allocation:

a) Resource Allocation:

P1 holds R1 & waits for R2
P2 holds R2 & waits for R3
P3 holds R3 & waits for R1

representation:

$R1 \rightarrow P1$

$P1 \rightarrow R2$

$R2 \rightarrow P2$

$P2 \rightarrow R3$

$R3 \rightarrow P3$

$P3 \rightarrow R1$

$P1 \rightarrow R2 \rightarrow P2 \rightarrow R3 \rightarrow P3 \rightarrow R1 \rightarrow P1$

b) Deadlock Detection:

Resource Allocation Graph contains a cycle & each resource has single instance.

c) strategy

one process terminated or its allocated resource R3 can be preempted.

5) CPU utilization - Multiprogramming level:

CPU utilization is percentage of time CPU is actively executing processes instead remaining idle.

Probability = 0.8

Degree = 4

Formula $\Rightarrow U = 1 - p^n$

a) Calculate CPU utilization for n = 6

$U = 1 - (0.8)^6$

$U = 1 - 0.262144$

$U = 0.737856$

CPU utilization = 73.79 %.

b) Calculate CPU utilization for n = 4

$U = 1 - (0.8)^4$

$U = 1 - 0.4096$

$U = 0.5904$

CPU utilization = 59.04 %.

c) Interpretation:

* For 4 process, CPU utilization is 59.0
* For 6 process, 73.79 %.
* 6 processes provide better system performance.

5) Producer - Consumer Problem

Synchronization problem where producer add items to shared buffer & consumers remove items from it.

Performance :

Semaphore values after each operation :

operation	Empty	Full	Mutex
Initial State	6	0	1
Producer 1	5	1	1
producer 2	4	2	1
producer 3	3	3	1
Consumer 1	4	2	1
Consumer 2	5	1	1

b) How semaphores prevent race conditions:

* Mutex semaphore allows only one process to access shared buffer at time.

* Empty semaphore keeps track of available empty slots in buffer.

* Full semaphore keeps track of filled slots available for consumers.

c) Effect of removing mutex semaphore:

If mutex semaphore is removed:

* Producers & consumers may access buffer simultaneously.

* Multiple processes can modify same buffer location at same time.

7) Process Synchronization:

Critical section is part of program where shared resource is accessed.

a) Explain how a race condition occurs:

Shared variable $X = 100$.

Two process increment $\times 1000$ times each.

b) Expected value of x:

Initial value $X = 100$

Process 1 increments $\times 1000$ times

Process 2 increments $\times 1000$ times.

Total increments $= 2000$

Final value of $X = 100 + 2000 = 2100$.

c) Synchronization solution:

Hardware Based: TSL / CAS

Software Based: Mutex lock

Thread Scheduling & Performance:

Multithreading is execution of multiple threads within a single process.

a)

Many to one	One to one
Many user threads are mapped to one kernel thread.	Each user thread is mapped to seperate kernel thread.
Blocking of 1 thread blocks all others.	One blocked thread doesn't affect other.

b) CPU utilization comparison:

Ans. No. of user-level threads = 40

CPU cores = 8

CPU time per thread = 12ms

Many to one:

CPU cores are underutilized

CPU utilization is lower because.

One to one:

Threads are distributed all 8 CPU cores

c) Recommended:

One-to-one multithreading model

It supports true parallelism.

Provides better CPU utilization.

a) Monitor synchronization:

Monitor is high-level synchronization mechanism that controls access to shared resources.

a) How Monitors ensure mutual exclusion,

Monitor contains shared data & procedures that operate on data, This prevents simultaneous access to shared resources.

b) Role of wait() & signal()

wait() → when a process cannot continue it calls wait().

signal() → when a resource become available, process calls signal()

c) Effect of omitting signal():

If signal() is not called after a print job completes:

Waiting process remain blocked,

Printer may become idle

Lead to Indefinite Starvation

Deadlock prevention vs Deadlock Avoidance

a) Deadlock prevention:

Deadlock prevention avoids deadlocks by eliminating one of four necessary conditions of deadlock.

b) Deadlock Avoidance:

It allows resource requests only if system remains in safe state.

Thus, avoidance prevents deadlocks by making safe allocation decisions.

c) Comparison:

Deadlock prevention	Deadlock Avoidance
Eliminates one of condition	Checks for a safe state.
Simpler to implement	More complex
Lower resource utilizer.	Better.
May reduce system performance	Improves.